Randomized Algorithm

Randomized Algorithm

- Generates a random number, r $\in$ (1,…,R)
- Make decisions based on r's value
- Recursive Algorithm
- Generate r in every level of recursion

Examples

- Monte Carlo Simulation
- Las Vegas Algorithm
- Atlantic City Algorithm

Origin of Monte Carlo Simulation

- What is the probability to get an even number from a random throw of dice?
- We can find the solution if the sample space is small and formulate the solution easily.

  In this case, possible sample space is 6, and when the outcome is 2,4 or 6, it satisfies the requirement. So, the result is : 3/6 = .5 (50%)

Origin of Monte Carlo Simulation

- Stanisław Ulam, thought about this at first, during his treatment.
- What is probability of solving a solitaire game from a random shuffled deck?
  Number of shuffle? 52! ≈ 10^68

- Total people in earth (around 10 billion): ≈ 10^10
- Seconds since big bang: (13.8 billion years): ≈ 10^17

Rather than finding the exact number: simulate a number games and find the number of wins, to get an approximation.

Monte Carlo Algorithm

- Approximate PI
- Finding expected number of rounds for a game

Approximation of PI

Geometric approach: Find the area of a circle having r radius and find the area of a square having 2r as the length of one side.

The ratio x = $\frac{\pi r^2}{4r^2}$

PI = 4*x

Approximation of PI (Monte Carlo Simulation)

Using randomization:

- Run a loop for a significant large number T
- Set a counter C = 0
- For each iteration:
  - Generate two **random numbers** (x, y) in the range of [-1, 1]
  - If $x^2 + y^2 <= 1$, *then C++*

(If the center of the circle at 0,0, then:

Distance of a point from the center = $\text{sqrt}[(x-0)^2 + (y-0)^2]$

$$\frac{\pi r^2}{4r^2} \approx \left(\frac{C}{T}\right)$$

Expected round of games

- Finding expected number rounds for a game, where the game ends if you loose 2 consecutive games. Given the win probability of a game is p, and p does not depend on previous outcome

Expected value

$$EV = \sum P(X_i) \times X_i$$

- EV (Project A) = [0.4 × $2,000,000] + [0.6 × $500,000] = $1,100,000
- EV (Project B) = [0.3 × $3,000,000] + [0.7 × $200,000] = $1,040,000

Round of games

- $E = (1+E)*p + 2*(1-p)(1-p) + (2+E)*(1-p)*p$

$E = (2 – P)/ (1-p)^2$

Round of games (Explanation)

2 Possible Scenarios for the first game:
- First game W
- First game L

For first game W, rest the of games are not affected, so total number of rounds will be: 1 + E. (Probablity: p)

If first game is L, then next game maybe W or L.

If 2nd game is also an L, then total number of rounds is: 2 (Probablity: (1-p)*(1-p)

If 2nd game is a W, then the total number of rounds will be 2 + E. (Probablity: (1-p)*p

Round of games (Monte Carlo)

- Run the simulation for a large number N

For each run:

Set two counters, nloss = 0, nround = 0

While (nloss != 2):

nround ++

Generate a **random number** r [0 to 1]

if r > p:

nloss++

else

nloss = 0

Save nround

- Find the average nround

Pros and Cons of Monte Carlo

- Analytical solution might be tricky and need in-depth knowledge of the problem domain, MC does not
- Less time to get the solution (most of the time)

- Solution design might be quick, but run time might be very high (round of games having winning probability closer to 1)
- No insight about the solution
- Reusing the solution for different value needs re-run
- Generalization might not be true always

Las Vegas Algorithm

- Guaranteed to get the result or does not find at all

- Example:
  - 8 Queen problem
  - Randomized Quick Sort

8 Queen Problem

- For $1 \leq k \leq 8$
- Find all possible valid location for the queen in the row k
- If no location found, report fail
- If more than one location found, choose one at random
- If k == 8, return solution

Comparison

- Monte Carlo algorithm:
  - Deterministic runtime (Fixed runtime)
  - Expected output (probably correct)
- Las Vegas algorithm:
  - Deterministic output (Always correct)
  - Expected runtime (probably faster)
- Atlantic City algorithm:
  - Probably faster
  - Probably correct

Quick Sort

```
quickSort(arr[], low, high)
{
    if (low < high)
    {
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);  // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
```

Quick Sort

```
partition (arr[], low, high)
{
    pivot = arr[high]; // Could be any random location
        i = low
        for (j = low; j <= high- 1; j++)
        {
            if (arr[j] < pivot)
                {
                    swap arr[i] and arr[j];
                i++;
                }
        }
        swap arr[i] and arr[high])
        return i
}
```

Randomized quick sort

```
partition_r (arr[], low, high)
{
    r = random value between low to high
        swap arr[r] and arr[high]
        return partition(arr, low, high)
}
```